

Api Integration and Data Migration

(DAY 3)

[Market-Place E-Commerce]

This report outlines the steps taken **to successfully integrate an external API with Sanity CMS, migrate product data, and display it on a web application.** The project involved configuring a backend script, setting up a schema in Sanity, and ensuring seamless API communication for dynamic data retrieval. The integration was performed using **Node.js, Axios, Sanity CMS, and Next.js.**

Project Objectives

- Fetch product data from an external API.
- Store and structure the data in Sanity CMS.
- Configure and execute a data migration script.
- Display the imported data dynamically on a Next.js-based frontend.

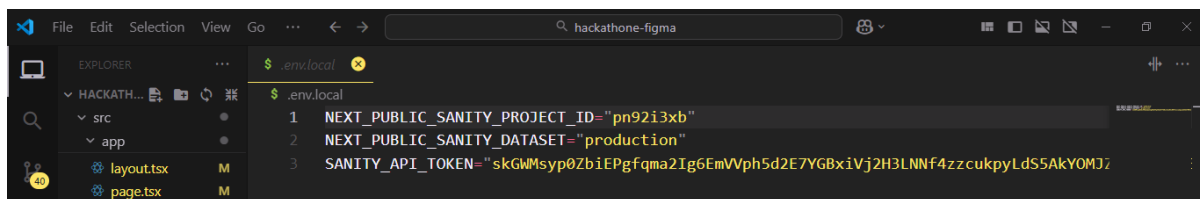
Steps Followed

Step 1: Sanity CMS Initialization

To begin, Sanity CMS was initialized in the project. This involved setting up the environment and configuring the authentication credentials.

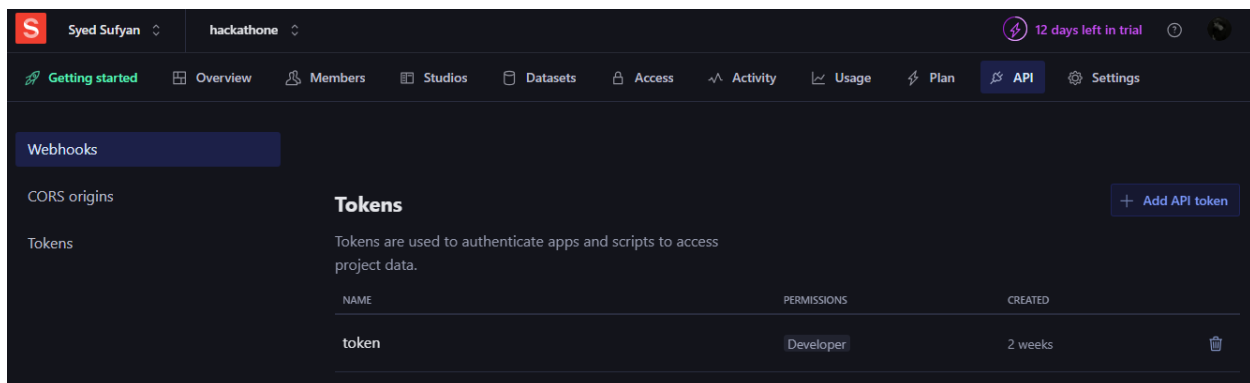
Tasks Completed:

- Installed **Sanity CLI** and created a new project.
- Generated an authentication token for secure API requests.
- Stored the token in a `.env` file to keep sensitive data secure.



A screenshot of the Visual Studio Code editor showing the `.env.local` file. The file contains the following configuration:

```
1 NEXT_PUBLIC_SANITY_PROJECT_ID="pn92i3xb"
2 NEXT_PUBLIC_SANITY_DATASET="production"
3 SANITY_API_TOKEN="skGwMsy0ZbiEPgfqma2Ig6EmVVph5d2E7YGBxiVj2H3LNNf4zzcukpyLdS5AkYOMJ"
```

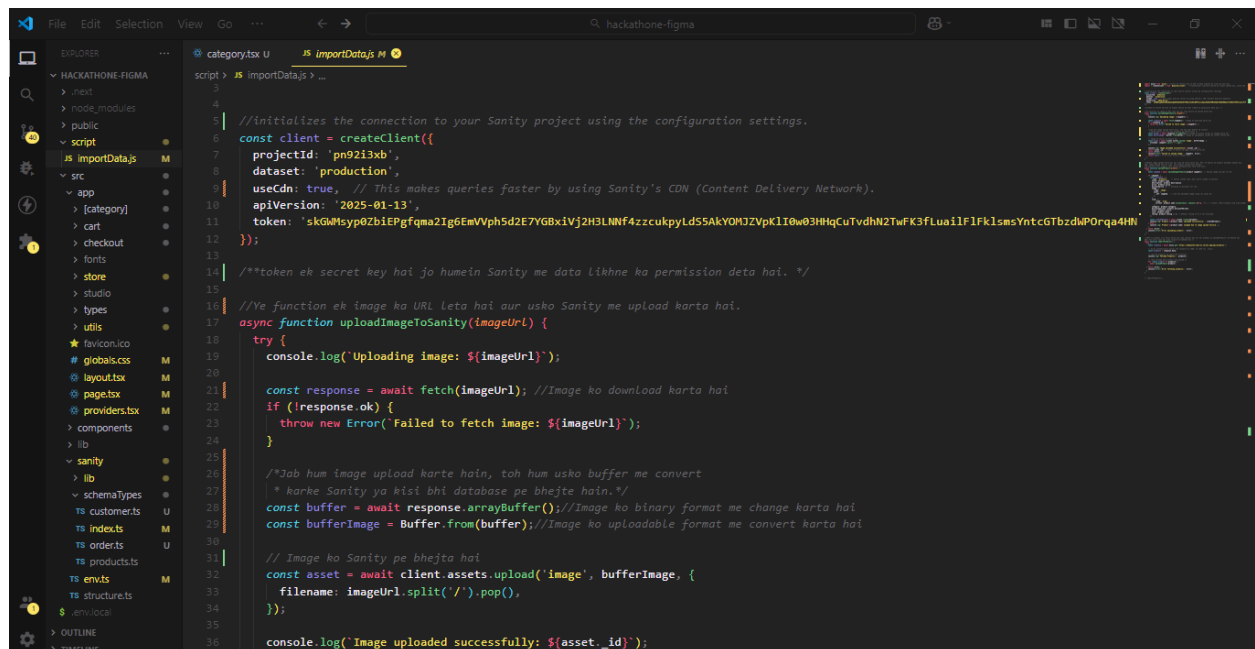


Step 2: Setting Up Import Script

A dedicated script was created to fetch data from the external API and send it to Sanity CMS.

Tasks Completed:

- Created a script folder in the project root.
- Added a new file named `importData.js`.
- Used **Axios** to fetch product data from the API.
- Mapped the received data to match the Sanity schema.
- Sent the processed data to Sanity CMS.



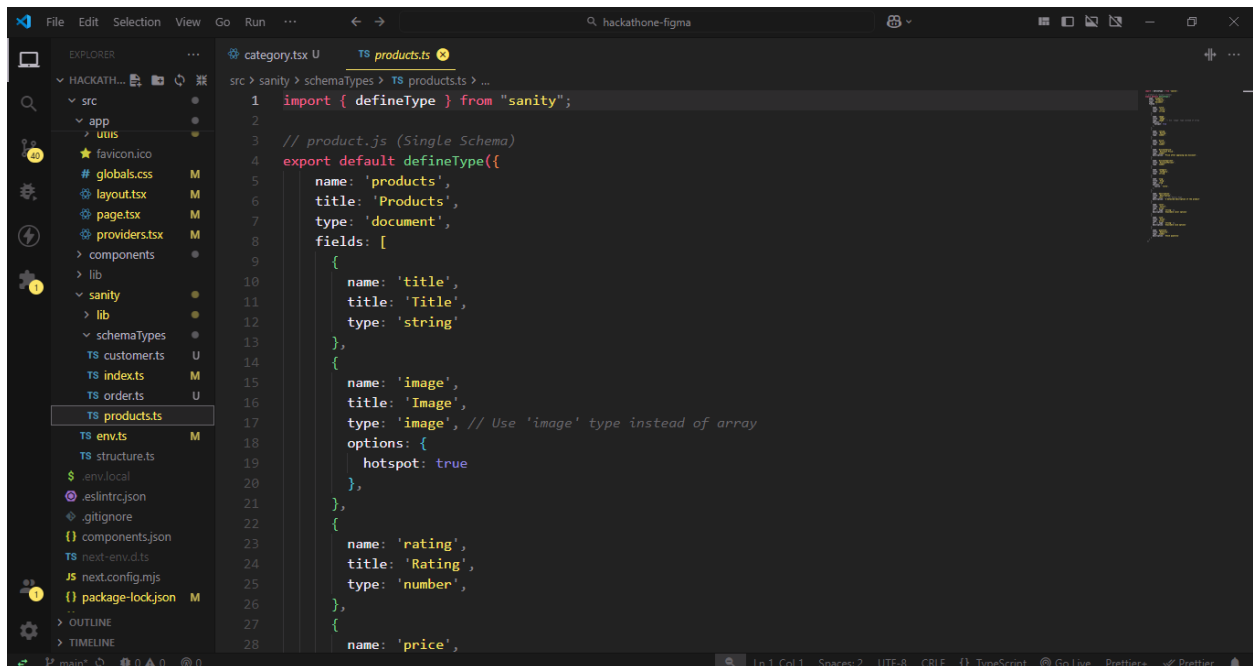
```
category.tsx U JS importData.js M
script > JS importData.js > ...
3
4
5 //initializes the connection to your Sanity project using the configuration settings.
6 const client = createClient({
7   projectId: 'pn92i3xb',
8   dataset: 'production',
9   useCdn: true, // This makes queries faster by using Sanity's CDN (Content Delivery Network).
10   apiVersion: '2025-01-13',
11   token: 'sk6Wt5yp0ZbiEPgfqma2Ig6EmVvph5d2E7YG8xiVj2H3LNNF4zzcukpyLd55AKYOMJZvpK1Iow03HhQCuTvdhN2TwFK3flua1iF1Fk1msYntcGTbzdWpQrqa4Hh',
12 });
13
14 /**token ek secret key hai jo humein Sanity me data likhne ka permission deta hai. */
15
16 //Ye function ek image ka URL leta hai aur usko Sanity me upload karta hai.
17 async function uploadImageToSanity(imageUrl) {
18   try {
19     console.log('Uploading image: ${imageUrl}');
20
21     const response = await fetch(imageUrl); //Image ko download karta hai
22     if (!response.ok) {
23       throw new Error('Failed to fetch image: ${imageUrl}');
24     }
25
26     //Job hum image upload karte hain, toh hum usko buffer me convert
27     //karke Sanity ya kisi bhi database pe bhejte hain.*/
28     const buffer = await response.arrayBuffer(); //Image ko binary format me change karta hai
29     const bufferImage = Buffer.from(buffer); //Image ko uploadable format me convert karta hai
30
31     // Image ko Sanity pe bhejta hai
32     const asset = await client.assets.upload('image', bufferImage, {
33       filename: imageUrl.split('/').pop(),
34     });
35
36     console.log('Image uploaded successfully: ${asset._id}');
```

Step 3: Creating & Updating the Sanity Schema

The Sanity schema was modified to align with the API's product structure.

Tasks Completed:

- Edited `product.ts` schema file.
- Defined relevant fields (e.g., name, price, description).
- Ensured the structure matched the API response format.



The screenshot shows a VS Code editor window with the file explorer on the left and the code editor on the right. The file explorer shows the project structure, including the `sanity` directory and the `products.ts` file. The code editor displays the content of `products.ts`, which defines a Sanity schema for products. The schema includes fields for `title`, `image`, `rating`, and `price`.

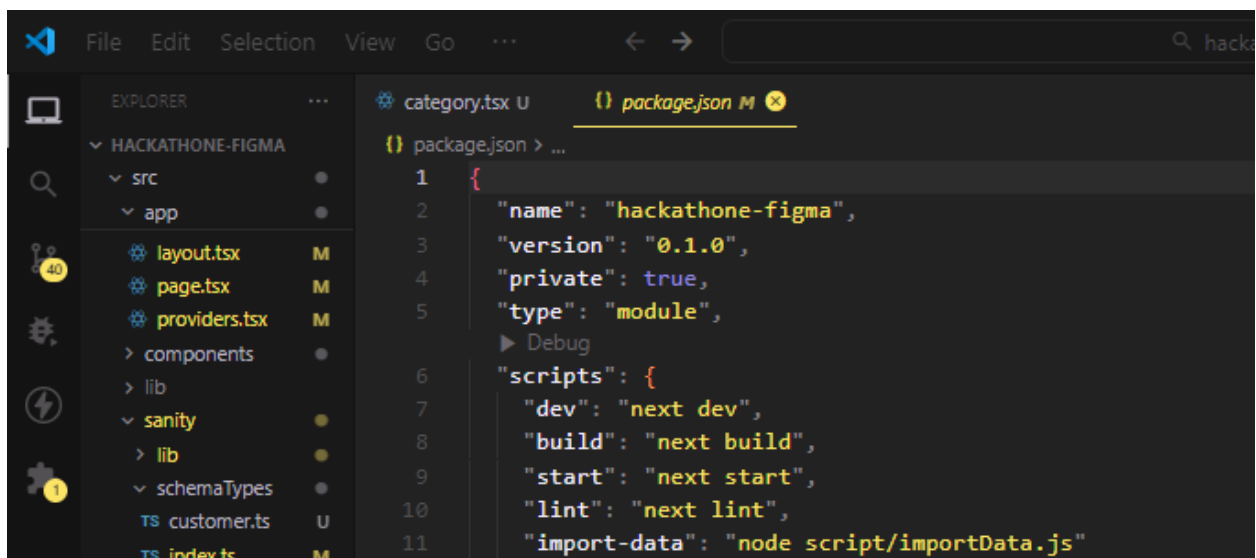
```
1 import { defineType } from "sanity";
2
3 // product.js (Single Schema)
4 export default defineType({
5   name: 'products',
6   title: 'Products',
7   type: 'document',
8   fields: [
9     {
10       name: 'title',
11       title: 'Title',
12       type: 'string'
13     },
14     {
15       name: 'image',
16       title: 'Image',
17       type: 'image', // Use 'image' type instead of array
18       options: {
19         hotspot: true
20       },
21     },
22     {
23       name: 'rating',
24       title: 'Rating',
25       type: 'number',
26     },
27     {
28       name: 'price',
```

Step 4: Configuring package.json

The package.json file was updated to support module imports and script execution.

Tasks Completed:

- Set "type": "module" to enable ES6 imports.
- Added a custom script to automate data import.



Step 5: Executing the Data Migration Process

Once the setup was complete, the following commands were executed to fetch and import the product data.

```
npm install @sanity/client axios dotenv  
npm run import-data  
node script/importData.js
```

After executing these commands, the product data was successfully migrated into Sanity CMS.

Step 6: Fetching and Displaying Data on the Website

Finally, the data stored in Sanity was retrieved and displayed on the website.

Tasks Completed:

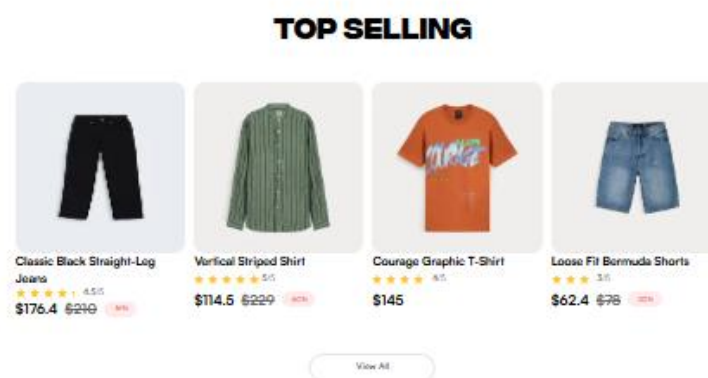
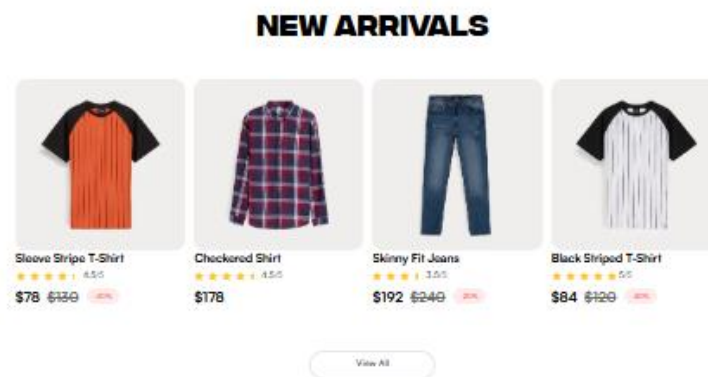
- Used **Sanity's client API** to fetch data.
- Rendered product details dynamically on the frontend.
- Ensured that UI updates automatically when new products are added.

File Edit Selection View Go Run ... hackathone-figma

page.tsx M topselling.tsx M

src > components > topselling.tsx > (x) Topselling

```
21 //data ko fetch kiya sanity database say grog query say query krkay
22 async function gettopsells() {
23   const query = await client.fetch(`* [_type == 'products' &&
24     title in ['Vertical Striped Shirt',
25       'Courage Graphic T-Shirt', 'Loose Fit Bermuda Shorts',
26       'Classic Black Straight-Leg Jeans']]
27 {
28   _id,
29   title,
30   "imageUrl": image.asset->url,
31   description,
32   price,
33   discountprice,
34   discountpercent,
35   rating,
36   'slug':slug.current,
37   category,
38 }`)
39
40   return query;
41
42 }
```



5. Conclusion

The API integration and data migration process was successfully completed. The external product data was fetched, structured, stored in Sanity CMS, and displayed dynamically on the website. This implementation ensures that new products can be added to the system seamlessly, making it a scalable and efficient solution for e-commerce platforms.

6. Submission Checklist

- ✓ Sanity CMS setup and environment variable configuration.
- ✓ Import script implementation and API data fetching.
- ✓ Schema definition and adjustments.
- ✓ Data migration execution and verification in Sanity.
- ✓ Product data displayed on the frontend.
- ✓ Screenshots and logs demonstrating successful implementation.

This report serves as comprehensive documentation of the **Sanity API Integration and Data Migration process**. It follows best practices in **CMS configuration, API integration, and frontend rendering**, ensuring a structured and maintainable system.

